

BACKEND DATABASE REALITY AUDIT

AŞAMA B0 — SALT-OKUMA GERÇEKLİK DOĞRULAMASI

Tarih: 2026-08-10 · Yöntem: canlı HTTP probe + repo kaynak okuma + veri kimliği karşılaştırması

Bu denetimde **hiçbir** kod, route, şema, migration, MCP tanımı veya deployment değiştirilmedi.

Gizli bilgiler (şifre, connection string, token, secret, API key) bu rapora **yazılmamıştır**.

BACKENDS

#	Ad	Adres	Tür	Rol (gözlenen)
A	Ana backend	https://canlifal.com	Web uygulaması + API katmanı (aynı origin altında / api/...)	Web sitesi, auth/oturum, kullanıcı, cüzdan, sosyal, kısa video, hikâye, mesaj, müzik, ödeme, TRTC imzası, SSE akışları
B	İkinci backend	https://canlifalapi.abacusai.app	Ayrı API servisi (Socket.IO + Redis'li)	Oyun odaları, PK sistemi, hediye savaşları/hedefleri, üyelik planları, canlı misafir listesi, gerçek zamanlı socket

Ana backend (A) — tespit edilen gerçekler

- **API kökü:** https://canlifal.com/api/... (ayrı bir API alt alan adı yok, /api/v1 öneki yok).
- **Health endpoint: YOK.** /api/health → 404 (yakalanmayan-uç yanıtı: ENDPOINT_NOT_FOUND). /health → 200 ama HTML sayfa döndürüyor, sağlıklı JSON'u değil.
- **Auth:** web oturum sistemi aktif (/api/auth/... uçları yanıt veriyor). Korumalı uçlar 401 döndürüyor (/api/me → 401 "Oturum açmanız gerekiyor"). Mobil giriş ucu mevcut (/api/auth/mobile-login → GET'te 405, yani route var, POST bekliyor).
- **Üretim veritabanı bağlantısı:** uygulama yapılandırmasında tek bir bağlantı değişkeni üzerinden. Kimlik (maskeli): **PostgreSQL uyumlu yönetilen veritabanı, örnek db-15af068d38 (db003.hosteddb.reai.io)**. Bağlantı dizesi ve kimlik bilgileri raporlanmadı.
- **Redis: KULLANILMIYOR.** Ortam yapılandırmasında Redis adresi yok, bağımlılıklarda Redis istemcisi yok. Bunun yerine lib/cache.ts içinde **süreç-içi (in-memory) TTL cache** var; arayüzü

Redis'e benzetilmiş ama veriler yalnız o sürecin belleğinde. Çok örnekli dağıtımda cache/durum tutarsızlığı riski.

• **Canlı probe sonuçları (A):**

Uç	Kod	Yorum
/	200	Site canlı (0.18 s)
/api/health	404	Sağlık ucu yok
/api/me	401	Auth katmanı çalışıyor
/api/chat/rooms	200	Gerçek veri
/api/games	200	Oyun kataloğu
/api/video-streams	200	streams: [] (o an yayın yok)
/api/live/gift-types	401	Auth korumalı
/api/wallet	401	Cüzdan A'da, auth korumalı
/api/messages	401	Mesajlaşma A'da
/api/social/posts	200	Sosyal veri A'da
/api/short-videos	200	Kısa video A'da
/api/stories	200	Hikâyeler A'da
/api/music/search	401	Müzik A'da, auth korumalı
/api/payments/methods	200	Ödeme A'da
/api/trtc/usersig	405	TRTC imza ucu A'da (POST bekliyor)
/api/notifications/stream	401	SSE ucu A'da, auth korumalı
/api/memberships	200	Üyelik (eski isimlendirme) A'da
/api/membership/plans	404	A'da yok
/api/pk/active	404	PK A'da yok
/api/games/rooms	404	Oyun odaları A'da yok

- **SSE:** ana backend envanterinde **19 akış ucu** tespit edilmişti (AŞAMA A). Canlı doğrulanan örnek: `/api/notifications/stream` → 401 (uç mevcut, oturum gerekli).

İkinci backend (B) — SADECE OKUNDU

- **A) DNS/HTTP canlı mı: EVET.** Kök / → 404 ama JSON hata gövdesi
(success:false, statusCode:404, message:"Cannot GET /") → servis ayakta. İlk istek **12.19 s** sürdü (soğuk başlatma göstergesi), sonraki istekler hızlı.
 - **B) Health cevap veriyor mu: EVET.**
 - /api/v1/health → 200
{ "status":"ok", "redis":"connected", "db":"connected", "instance":"canlifal-api-1", ... }
 - /health → 200 { "main":true, "game":true, "redis":true, "database":true }
 - **C) Production ortamı mı: EVET.** Örnek adı canlifal-api-1 , gerçek üretim verisi döndürüyor (5 sohbet odası, 25 hediye, aktif PK maçları, gerçek oyun odaları), Socket.IO canlı.
 - **D) Gerçekten çalışan route'lar:** aşağıdaki ## ACTIVE SECOND BACKEND FEATURES bölümü.
 - **E) Hangi veritabanı:** doğrudan okunamadı (ortam erişimi yok). **Veri kimliği karşılaştırmasıyla A ile aynı veritabanı olduğu kanıtlandı** → ## DATABASE .
 - **F) Hangi Redis:** health yanıtı redis: connected diyor; Redis örneğinin kimliği/adresi **NOT VERIFIED** (dışarıdan okunamaz).
 - **G) Şema/model yapısı:** üretimdeki şemanın kaynağı elimizde **YOK** (aşağıdaki BLOCKER 1). Repo'daki api/ şeması 36 model içeriyor; üretim yanıt gövdeleri repo kodundan farklı → repo şeması üretimi temsil etmiyor olabilir.
-

LIVE STATUS

Kontrol	Sonuç	Kanıt
canlifal.com canlı	YES	/ → 200, 0.18 s
canlifal.com health ucu var	NO	/api/health → 404
canlifalapi.abacusai.app canlı	YES	/api/v1/health → 200
İkinci backend health OK	YES	status:ok, db:connected, redis:connected
İkinci backend üretimde kullanılıyor	YES	Gerçek veri + Socket.IO + Flutter varsayılan yönlendirmesi
İkinci backend Socket.IO canlı	YES	/socket.io/?EIO=4&transport=polling → 200, sid üretti, upgrades: ["websocket"], pingInterval 25000
Ana backend Redis kullanıyor	NO	Bağımlılık yok, ortam değişkeni yok, süreç-içi cache
İkinci backend Redis kullanıyor	YES	health redis: connected

Soğuk başlatma notu: ikinci backend'in ilk isteği 12 saniyeyi aştı. Mobil uygulamada uzun bekleme / zaman aşımı hatalarının olası kaynağıdır.

DATABASE

BACKEND A DB: PostgreSQL uyumlu yönetilen veritabanı – örnek
 "db-15af068d38" (db003.hosteddb.reai.io)
 [kimlik bilgileri ve bağlantı dizesi kasıtlı olarak raporlanmadı]
 BACKEND B DB: Doğrudan okunamadı (ortam erişimi yok). Health ucu "db: connected" diyor.
 Kimlik, veri karşılaştırmasıyla dolaylı olarak A ile AYNI tespit edildi.

Kanıt yöntemi

Her iki backend'den **aynı kaynaklar** çekildi ve döndürdükleri birincil anahtarlar karşılaştırıldı. Kullanılan kimlikler çakışma olasılığı pratikte sıfır olan rastgele kimliklerdir; aynı kimliğin iki bağımsız veritabanında bulunması mümkün değildir.

Kaynak	A'dan dönen kimlik	B'den dönen kimlik	Eşleşme
Sohbet odası (sohbet)	cmokyb9o9007iod09-gi6pb1tb	cmokyb9o9007iod09-gi6pb1tb	AYNI
Oyun (fal-carkı / "Fal Çarkı")	cmokscun-b006spnkor6o1p8tg	cmokscun-b006spnkor6o1p8tg	AYNI
Kullanıcı (Admin)	cmoks-cu2y0000pnko11nc-tqw5	Aynı kimlik B'nin oyun odası yanıtında oyuncu alanında	AYNI

Ayrıca B'den /api/chat/rooms → 5 oda, /api/gifts → 25 hediye (canlifal_1 , canlifal_5 , gul , kalp , yildiz ...) — tümü üretim verisi.

Gerçeklik tablosu

Soru	Cevap	Gerekçe
AYNI DATABASE	YES	Paylaşılan birincil anahtarlar (yukarıdaki tablo)
AYNI SCHEMA	NO	Aynı veritabanına iki ayrı şema tanımı bakıyor: ana şema 196 model, ikinci backend repo şeması 36 model. Ortak 13 modelde alan sayıları farklı (ör. User 123 vs 38 alan, Conversation 8 vs 11, PKBattle 19 vs 21, SocialPost 17 vs 18). Ayrıca ikinci şemada ana şemada bulunmayan 23 model var.
AYNI USER MODEL	NO	Aynı tablo, farklı model tanımı. İkinci taraf avatarUrl , coins , displayName , coverUrl gibi alanlar tanımlıyor; ana taraf 100'ü aşkın ek alan tanımlıyor.
AYNI BALANCE / JETON VERİSİ	NOT VERIFIED	Aynı veritabanı olduğu için aynı tabloya yazıldığı çok muhtemel; ancak doğrudan doğrulanmadı. Üretimde cüzdan uçları yalnız A'da yanıt veriyor (/api/wallet A → 401 auth, B → 404), yani jeton işlemleri fiilen B üzerinden geçmiyor görünüyor.
AYNI LIVE DATA	YES	Her iki backend /api/video-streams için aynı (boş) sonucu döndürdü; aynı veritabanı zaten kanıtlı.
AYNI SOCIAL DATA	NOT VERIFIED	B'de /api/social/posts → 404. Sosyal veri B üzerinden servis edilmiyor; karşılaştırma yapılamadı.
AYNI CONVERSATION DATA	NOT VERIFIED	B'de /api/messages → 404. Mesaj verisi B üzerinden servis edilmiyor; karşılaştırma yapılamadı.

DATABASE SONUCU: SAME DATABASE (farklı şema tanımlarıyla erişilen tek bir veritabanı).

SCHEMA

Ölçüm	Ana backend	İkinci backend (repo kodu)
Model sayısı	196	36
Ortak model	13	13
Yalnız kendi tarafında	183	23
Kodda hiç sorgulanmayan model	36	—
Migration geçmişi	Ana uygulamanın kendi geçmişi	Ayrı, 8 migration

Risk: Tek bir veritabanı, iki bağımsız migration geçmişi tarafından yönetiliyor. Taraflardan biri şema değişikliği uygularsa diğer tarafın modeli sessizce bozulabilir. Hangi tarafın otoritatif olduğu **belirsiz**.

Üretim şeması doğrulanamadı: aşağıdaki BLOCKER 1 nedeniyle üretimdeki ikinci backend'in gerçek şema kaynağı elimizde yok; yukarıdaki 36 model repo gerçeğidir, üretim gerçeği **NOT VERIFIED**.

FLUTTER ROUTING

Base URL tanımları (`mobile/lib/core/config/env.dart`)

Değişken	Varsayılan	Sınıf
<code>API_BASE_URL</code>	<code>https://canlifal.com</code>	CANLIFAL.COM
<code>WEB_ORIGIN</code>	<code>https://canlifal.com</code>	CANLIFAL.COM
<code>GAMES_API_BASE_URL</code>	<code>https://canli-falapi.abacusai.app</code>	CANLI-FALAPI.ABACUSAI.APP
<code>GATEWAY_API_BASE_URL</code>	<code>' ' (boş)</code>	OTHER — fiilen devre dışı

- `useSplitGamesApi` → oyun adresi boş değilse ve ana adresten farklıysa **true**. Varsayılan derlemede **TRUE** → uygulama **iki backend'i birden** kullanıyor.
- `gatewayApiBaseUrl` yalnızca 502/503/504 sonrası tek bir yeniden denemede kullanılıyor; varsayılanı boş olduğu için üretimde etkin değil.

Yönlendirme kuralı (`mobile/lib/core/network/api_backend_router.dart`)

Her istek path'i şu kurallarla ikinci backend'e (**GAME**) gönderiliyor, kalan her şey ana backend'e (**MAIN**):

1. `/api/pk...` ile başlayan her şey
2. `/api/gifts/battles...` , `/api/gifts/goals...`
3. `/api/membership...` ile başlayan her şey
4. `/api/live/pk/active...` , `/api/live/guest/...`
5. `/api/games/rooms` , `/api/games/auto-match` , `/api/games/play` , `/api/games/room...`

Sınıflandırma sonucu (Flutter'ın çağırdığı 521 farklı path)

Hedef	Adet
CANLIFAL.COM (MAIN)	463
CANLIFALAPI.ABACUSAI.APP (GAME)	58
OTHER	0 (gateway boş)
UNKNOWN	0 (router tüm path'leri deterministik çözüyor)

İkinci backend'e giden 58 path

<code>/api/games/auto-match</code>	<code>/api/pk</code>
<code>/api/games/play</code>	<code>/api/pk/</code>
<code>/api/games/room</code>	<code>/api/pk/\$matchId</code>
<code>/api/games/room/\$roomId</code>	<code>/api/pk/\$matchId/cancel</code>
<code>/api/games/room/\$roomId/chat</code>	<code>/api/pk/\$matchId/end</code>
<code>/api/games/room/\$roomId/join</code>	<code>/api/pk/\$matchId/events</code>
<code>/api/games/room/\$roomId/viewers</code>	<code>/api/pk/\$matchId/respond</code>
<code>/api/games/room/abc123</code>	<code>/api/pk/\$matchId/seats/join</code>
<code>/api/games/room/abc123/join</code>	<code>/api/pk/\$matchId/seats/kick</code>
<code>/api/games/rooms</code>	<code>/api/pk/\$matchId/seats/leave</code>
<code>/api/gifts/battles</code>	<code>/api/pk/\$matchId/start</code>
<code>/api/gifts/battles/\$id</code>	<code>/api/pk/\$matchId/stream</code>
<code>/api/gifts/goals</code>	<code>/api/pk/active</code>
<code>/api/live/guest/</code>	<code>/api/pk/admin/\$matchId/force-end</code>
<code>/api/live/guest/list</code>	<code>/api/pk/admin/\$matchId/force-kick/\$userId</code>
<code>/api/live/pk/active</code>	<code>/api/pk/admin/ban</code>
<code>/api/membership</code>	<code>/api/pk/admin/bans</code>
<code>/api/membership-badges</code>	<code>/api/pk/admin/unban/\$userId</code>
<code>/api/membership/packages</code>	<code>/api/pk/battles</code>
<code>/api/membership/plans</code>	<code>/api/pk/battles/\$battleId</code>
<code>/api/memberships</code>	<code>/api/pk/battles/\$battleId/accept</code>
<code>/api/memberships/packages</code>	<code>/api/pk/battles/\$battleId/end</code>
<code>/api/memberships/purchase</code>	<code>/api/pk/battles/\$battleId/reject</code>
<code>/api/pk/cm123/stream</code>	<code>/api/pk/history</code>
<code>/api/pk/leaderboard</code>	<code>/api/pk/me/history</code>
<code>/api/pk/me/invites</code>	<code>/api/pk/me/matches</code>
<code>/api/pk/me/stats</code>	<code>/api/pk/request</code>
<code>/api/pk/room</code>	<code>/api/pk/stats/\$userId</code>
<code>/api/pk/stats/:userId</code>	<code>/api/pk/{id}/events</code>

Canlı doğrulama — yönlendirme doğru mu?

Path	İkinci backend	Ana backend	Değerlendirme
/api/pk/active	200	404	Yönlendirme doğru
/api/pk/leaderboard	200	404	Doğru
/api/gifts/battles	200	404	Doğru
/api/gifts/goals	200	404	Doğru
/api/membership/plans	200	404	Doğru
/api/games/rooms	200	404	Doğru
/api/live/guest/list	200	404	Doğru
/api/memberships	404	200	 YANLIŞ YÖN-LENDİRME — kural /api/membership ön ekiyle eşleştiği için /api/memberships de ikinci backend'e gidiyor, ama bu uç yalnız ana backend'de var
/api/pk/history	404	404	 Hiçbir backend'de yok
/api/games/auto-match	404	404	 Hiçbir backend'de yok

Eksik uçlar (AŞAMA A ölçümü, B0'da teyit edildi)

Flutter'ın çağırdığı **187 path** ana backend'de yok. Bunlardan **67'si** ikinci backend repo kodunda tanımlı, **120'si hiçbir yerde tanımlı değil** → üretimde 404 riski. B0'da rastgele örneklerle bu risk canlı olarak doğrulandı (/api/pk/history , /api/games/auto-match).

ACTIVE SECOND BACKEND FEATURES

Canlı olarak **200** döndüğü doğrulanan, yani üretimde fiilen iş gören özellikler:

Özellik	Uç	Kanıt
Sağlık/izleme	/api/v1/health , /health	status:ok , instance:canlifal-api-1
Oyun kataloğu	/api/games	Gerçek oyun listesi
Oyun odaları	/api/games/rooms	Gerçek oda listesi (okey101 vb.)
Sohbet odaları	/api/chat/rooms	5 gerçek oda
Hediye kataloğu	/api/gifts	25 hediye
Hediye savaşı / hedefi	/api/gifts/battles , /api/gifts/goals	200
PK sistemi	/api/pk/active , /api/pk/leaderboard , /api/live/pk/active	Aktif PK maçları döndü
Üyelik planları	/api/membership/plans	Plan listesi
Canlı misafir listesi	/api/live/guest/list	200
Yayın listesi	/api/video-streams	200 (o an boş)
Bildirimler	/api/notifications	401 → uç var, auth korumalı
Gerçek zamanlı soket	/socket.io/	200, oturum kimliği üretildi, websocket yükseltmesi destekli
Redis	health	connected

UNUSED BACKEND FEATURES

İkinci backend'de repo kodunda tanımlı ama canlıda 404 (üretimde yok)

/api/v1/me · /api/me · /api/health · /api/auth/me · /api/auth/mobile-login · /api/wallet · /api/short-videos · /api/social/posts · /api/trtc/usersig · /api/livekit/token · /api/music/search · /api/messages · /api/stories · /api/payment/config · /api/pk/history · /api/games/auto-match

→ Bu uçların **tümü** ana backend'de mevcut ve çalışıyor. Yani ikinci backend'in bu alanlardaki kodu üretimde **kullanılmıyor**.

LiveKit

`/api/livekit/token` canlıda **404**. LiveKit üretimde kullanılmıyor → yalnız kod kalıntısı.

Agora

Her iki backend'de **0** referans, mobil bağımlılıklarda paket yok; yalnız 23 isimlendirme kalıntısı. **Üretimde kullanılmıyor.**

Ana backend

- **42 route** hiçbir istemciden (web, mobil, ikinci backend, MCP) referanslanmıyor.
 - **36 veri modeli** kod tarafından hiç sorgulanmıyor.
 - **19 SSE ucu** mevcut; yalnız bir kısmı mobil tarafından tüketiliyor (kesin tüketim haritası AŞAMA B'de çıkarılacak).
-

DATA CONFLICT RISK

#	Risk	Şiddet	Açıklama
1	Tek veritabanı, iki şema tanımı	● YÜKSEK	İki bağımsız migration geçmişi aynı veritabanını yönetiyor. Bir taraf sütun eklerse/değiştirirse diğer taraf çalışma zamanında kırılabılır. Hangi tarafın otorite olduğu belirsiz.
2	User modeli iki farklı tanım	● YÜKSEK	Aynı tablo, 123 alanlı ve 38 alanlı iki ayrı tanım. İkinci taraf bilmediği zorunlu alanlara yazmadan kayıt oluşturursa tutarsız kullanıcı kayıtları oluşur.
3	Farklı cache/durum katmanı	● YÜKSEK	Ana backend süreç-içi bellek cache'i, ikinci backend Redis kullanılıyor. Oda durumu, koltuk, sayaç gibi paylaşılan durumlar iki ayrı yerde tutuluyor → kullanıcıya farklı görünme riski.
4	Jeton/bakiye yazımı	● ORTA	Aynı veritabanı üzerinde iki servis de bakiye yazabilir; ortak kilit/transaction stratejisi doğrulanmadı. Üretimde cüzdan uçlarının yalnız ana backend'de olması riski şu an sınırlıyor.
5	Üretim kodu ile repo kodu ayrılmış	● YÜKSEK	Ayrıntı: BLOCKER 1. Elimizdeki envanter üretimi temsil etmiyor olabilir.
6	120 tanımsız uç	● ORTA	

#	Risk	Şiddet	Açıklama
			Mobil uygulamanın çağırdığı 120 path hiçbir backend'de yok → sessiz 404, boş ekran, hatalı davranış.
7	Yanlış yönlendirme (/api/memberships)	🟡 ORTA	Mobil bu ucu ikinci backend'e gönderiyor (404), oysa ana backend'de 200 dönüyor.
8	Soğuk başlatma gecikmesi	🟡 DÜŞÜK	İkinci backend ilk istekte 12+ saniye. Zaman aşımı hataları.

RECOMMENDATION

Sonuç: B) SECOND BACKEND IS ACTIVE

İkinci backend ölü değildir. Üretimde şu özellikleri fiilen o servis karşılıyor: **oyun odaları, PK sistemi (tüm alt uçlar), hediye savaşları/hedefleri, üyelik planları, canlı misafir listesi ve gerçek zamanlı soket bağlantısı**. Mobil uygulama varsayılan derlemesinde bu 58 path'i doğrudan ikinci backend'e gönderiyor.

Veritabanı sonucu: SAME DATABASE

İki backend aynı veritabanına bakıyor (paylaşılan birincil anahtarlarla kanıtlandı), ancak **farklı iki şema tanımı** üzerinden. Dolayısıyla **veri taşıma (migration) gerekmiyor**; gereken şey **şema tanımının tekilleştirilmesidir**.

AŞAMA B için önerilen hedef

Tek backend = <https://canlifal.com>. Ancak bu bugün uygulanamaz; ikinci backend üretimde canlı iş görüyor. Önerilen sıra (hiçbiri bu aşamada uygulanmadı):

- Üretim kaynak kodunun elde edilmesi** (BLOCKER 1 çözülmeden hiçbir taşıma güvenli değildir).
- Şema otoritesinin belirlenmesi**: tek bir şema tanımı otoritatif ilan edilir, ikinci tarafın şeması salt-okuma türetilmiş görünüme dönüştürülür. Migration çalıştırma yetkisi tek tarafa verilir.
- Sözleşme dondurma**: ikinci backend'in canlı 200 dönen uçları için yanıt/hata biçimleri kayıt altına alınır (AŞAMA B'nin API sözleşmesi girdisi).
- Özellik-özellik taşıma sırası** (düşükten yükseğe risk): `membership/plans` → `gifts/goals` → `gifts/battles` → `live/guest` → `pk/*` → `games/room*` → Socket.IO gerçek zamanlı katman (en yüksek risk, Redis bağımlı).
- Realtime birleştirme**: ana backend Redis'siz olduğu için Socket.IO taşınmadan önce ana tarafa paylaşımlı durum katmanı gerekir. Aksi halde oda durumu bozulur.

6. **Mobil düzeltmeleri:** `/api/memberships` yanlış yönlendirmesi ve 120 tanımsız uç, sözleşme netleştikten sonra tek `ApiClient` içinde ele alınır.

Bu aşamada silinen, taşınan, değiştirilen hiçbir şey yoktur.

AŞAMA B READY (YES/NO)

NO — koşullu

API sözleşmesi ve yanıt/hata standardı çalışmasına **ana backend için** hemen başlanabilir. Ancak **iki backend'i kapsayan tek sözleşme** BLOCKER 1 çözülmeden yazılamaz: canlı ikinci backend'in gerçek kaynak kodu ve şeması elimizde yok, elimizdeki envanter onun üretim gerçeğini yansıtmıyor.

Kapsam	Hazır mı
Ana backend API sözleşmesi + yanıt/hata standardı	YES
İkinci backend'i kapsayan birleşik sözleşme	NO (BLOCKER 1)
Şema tekilleştirme	NO (BLOCKER 2)
Realtime tekilleştirme	NO (BLOCKER 3)
Mobil tek <code>ApiClient</code>	NO (BLOCKER 1 ve 4)

BLOCKER

BLOCKER 1 — Üretimdeki ikinci backend'in kaynak kodu elimizde yok. ●

Canlı `https://canlifalapi.abacusai.app`, mobil deposundaki `api/` klasörünün kodu **değildir**. Kanıt:

- Repo kodu health yanıtını `{success:true, data:{...}}` sarmalıyla döndürüyor; canlı servis düz `{status, redis, db, instance}` döndürüyor.

- Repo'da bağlanmış olan `/api/wallet`, `/api/messages`, `/api/trtc/usersig`, `/api/livekit/token` uçları canlıda 404.

→ **Sonuç:** AŞAMA A'da ölçülen 276 uç repo gerçeğidir, üretim gerçeği değildir. Gerekli: üretim deposuna/deployment kaynağına erişim.

BLOCKER 2 — Tek veritabanı, iki otoritesiz şema tanımı. ●

Hangi tarafın migration geçmişinin otoritatif olduğu belirsiz. Karar verilmeden şema birleştirme yapılamaz.

BLOCKER 3 — Paylaşılan durum katmanı yok. ●

Ana backend Redis kullanmıyor (süreç-içi bellek), ikinci backend Redis kullanıyor. Gerçek zamanlı özellikler tekilleştirilmeden önce ana tarafta paylaşımlı durum katmanı kararı gerekiyor.

BLOCKER 4 — 120 uç hiçbir backend'de tanımlı değil.

Mobil uygulamanın çağırdığı 120 path ne ana ne ikinci backend'de bulunuyor. Bunların "kaldırılacak mı, yazılacak mı" kararı AŞAMA B sözleşmesinin ön koşuludur.

NOT VERIFIED kalanlar (dürüslük kaydı):

- İkinci backend'in veritabanı kimliği doğrudan okunamadı (yalnız veri kanıtıyla eşleştirildi).
- İkinci backend'in Redis örneğinin kimliği/adresi.
- Jeton/bakiye verisinin iki taraftan da yazılıp yazılmadığı.
- Sosyal ve mesaj verisinin karşılaştırması (ikinci backend'de bu uçlar 404).
- Üretimdeki ikinci backend'in gerçek şema/model yapısı.
- Fiziksel cihaz gerektiren TRTC / kamera / mikrofon uçtan uca testi → **NOT PERFORMED**.